Mass Dynamics API — End-to-End Tour

A guided walk-through of the `md-python` client. Sections 1–5 hit live data. Section 6 demonstrates a full proteomics pipeline — runnable when you have your own upload to feed it, dry-run by default so the recording stays tight. Section 7 is the **workspaces beta** — programmatic dashboards.

Pre-reqs: a `.env` in the project root with `MD_AUTH_TOKEN` (and optional `MD_API_BASE_URL`).

1 · Connect

One line. The client reads `MD_AUTH_TOKEN` and `MD_API_BASE_URL` from the environment / `.env`.

In [1]:
```
from md_python import MDClient

client = MDClient(version="v2")
print(f"connected to {client.base_url}")
print(f"health: {client.health.check()}")
```

```
connected to https://dev.massdynamics.com/api
health: {'status': 'ok'}
```

2a · Browse uploads

Uploads are the parent records — they hold sample metadata and one or more datasets each. Free-text searchable.

In [2]:
```
import json

result = client.uploads.query(search="CKD", page=1)
print(f"uploads matching 'CKD': {len(result['data'])}")
for u in result['data'][:5]:
    print(f"  {u['id']}  {u['name']!r}")
```

```
uploads matching 'CKD': 10
  6db6e841-a1ef-4d74-b920-c6be7b581fb1  'CKD Demo 2025'
  5ea6cee9-5e48-4a6e-bc7e-09dbf8163458  '2025-07-23 CKD Demo'
  60cb23ad-f459-4597-8ed7-54d4a7a4edea  'CKD Proteomics - Stage 1/3/5 vs Con
trol'
  aac8a468-cade-45f6-b0bd-f363afd4db15  'CKD-Test-dataset-for-heatmaps'
  096fc685-b60d-4f41-b0be-fc2a637269eb  'Test-ckd'
```

2b · Browse datasets

Datasets are typed `INTENSITY` (raw or normalised), `PAIRWISE` (limma output), `ANOVA`, `DOSE_RESPONSE`. `query` filters on type, state, parent upload, or free text.

In [3]:
```
page = client.datasets.query(
    type=["INTENSITY"], state=["COMPLETED"], search="CKD", page=1,
)
print(f"COMPLETED INTENSITY datasets matching 'CKD': {page['pagination']['to
for d in page['data'][:5]:
    print(f"  {d['id']}  {d['name']!r}")
```

```
COMPLETED INTENSITY datasets matching 'CKD': 6
  fb3e4141-926e-4b97-bf20-e29d6d818570  'CKD demo imputation'
  1e97e2ae-1ad6-4d3b-a772-f2d496d152fa  'CKD Demo 2025'
  26b463b7-1ea6-4c73-9bbe-fdaea870e596  '2025-07-23 CKD Demo — MNAR Imputed'
  da331a86-0a65-4c60-bbbb-af7f5c78e6f4  '2025-07-23 CKD Demo'
  a2b30a56-25df-42b0-b32c-9d72d491c0d6  'CKD Proteomics - NI (skip norm, MNA
R imputation)'
```

3 · Inspect a dataset

The DATASET id is what every analysis and visualisation consumes; the upload id is the parent record.

In [4]:
```
# CKD Demo 2025 — a public proteomics demo workspace on the platform.
UPLOAD_ID = "6db6e841-a1ef-4d74-b920-c6be7b581fb1"
DATASET_ID = "1e97e2ae-1ad6-4d3b-a772-f2d496d152fa"

ds = client.datasets.get_by_id(DATASET_ID)
print(f"name : {ds.name!r}")
print(f"type : {ds.type}")
print(f"state: {ds.state}")
```

```
name : 'CKD Demo 2025'
type : INTENSITY
state: COMPLETED
```

4 · Search entities (proteins, genes, peptides)

Find a specific entity by gene symbol or accession. Server-side — you never download the underlying data.

In [5]:
```
results = client.entities.query(keyword="APOE", dataset_ids=[DATASET_ID])
for hit in results["results"]:
    print(f"dataset {hit['dataset_id']} ({hit['entity_type']})")
    for item in hit["items"]:
        print(f"  {item['GeneNames']!s:<8}  ProteinIds={item['ProteinIds']}"
```

```
dataset 1e97e2ae-1ad6-4d3b-a772-f2d496d152fa (protein)
  APOE      ProteinIds=P02649
```

5 · List analysis pipelines

Every pipeline you can launch — normalisation/imputation, pairwise comparison, ANOVA, dose-response — has a slug and a parameter schema.

In [6]:
```
jobs = client.jobs.list()
for j in jobs:
    print(f"  {j['slug']:<40}  {j['name']}")
```

```
normalisation_imputation                 normalisation imputation
custom_r_transform_intensities           Custom R Transform Intensities
flexicomparisons_dev_pairwise            FlexiComparisons-Dev (Pairwise)
dose_response_aggregate                  dose response aggregate
intensity                                intensity
md_hello_world                           md hello world
pairwise_comparison                      pairwise comparison
camera_gsea                              CAMERA GSEA
dose_response                            dose response
demo_flow                                demo_flow
wgcna                                    WGCNA
md_dataset_custom_r                      md dataset custom r
initial_job                              initial job
knn_tn_imputation                        kNN-TN Imputation
basic_r_to_say_hi                        basic_r_to_say_hi
jnj_dose_response                        jnj dose response
anova                                    ANOVA
```

---

6 · Full pipeline flow

An end-to-end proteomics analysis: **upload → normalise/impute → pairwise comparison → ANOVA**. Each step is a single API call.

The cells below are runnable, but `DRY_RUN = True` by default — they print the call shape and skip execution so the demo stays tight (real pipelines take 10–40 minutes). Flip the toggle when you have your own data.

In [7]:
```
DRY_RUN = True   # flip to False to actually create + run

# Placeholder ids the dry-run cells reference. Replace with your own when
# you flip DRY_RUN off.
DRY = {
    "upload_id":            "<upload-uuid>",
    "intensity_dataset_id": "<intensity-uuid>",
    "ni_dataset_id":        "<ni-output-uuid>",
    "pairwise_dataset_id":  "<pairwise-uuid>",
    "anova_dataset_id":     "<anova-uuid>",
}

def call(label, fn, *args, **kwargs):
    if DRY_RUN:
        print(f"[dry-run] {label}")
        return None
```

```
    print(f"[live] {label}")
    return fn(*args, **kwargs)
```

6a · Create an upload

Submit a proteomics dataset (DIA-NN, MaxQuant, Spectronaut, MD-format, etc.) along with the sample metadata. Returns the upload id; ingestion runs in the background.

In [8]:
```
from md_python.models import Upload, SampleMetadata, ExperimentDesign

upload = Upload(
    name="Demo run \u2014 pipeline tour",
    source="diann_tabular",
    file_location="/path/to/your/report.pg_matrix.tsv",
    filenames=["report.pg_matrix.tsv"],
    sample_metadata=SampleMetadata(data=[
        ["sample_name", "condition"],
        ["S1",          "control"],
        ["S2",          "control"],
        ["S3",          "treatment"],
        ["S4",          "treatment"],
    ]),
    experiment_design=ExperimentDesign(data=[
        ["filename",                "sample_name"],
        ["report.pg_matrix.tsv",   "S1"],
        ["report.pg_matrix.tsv",   "S2"],
        ["report.pg_matrix.tsv",   "S3"],
        ["report.pg_matrix.tsv",   "S4"],
    ]),
)

upload_id = call(
    "client.uploads.create(upload)  \u2192  upload_id",
    client.uploads.create, upload, background=True,
) or DRY["upload_id"]
print(f"  upload_id = {upload_id}")
```

```
[dry-run] client.uploads.create(upload)  →  upload_id
  upload_id = <upload-uuid>
```

6b · Wait for the upload to complete

Polls until the upload reaches `COMPLETED`. Real ingestion typically takes a few minutes for DIA / DDA reports.

In [9]:
```
call(
    f"client.uploads.wait_until_complete({upload_id!r})",
    client.uploads.wait_until_complete, upload_id,
)
```

```
[dry-run] client.uploads.wait_until_complete('<upload-uuid>')
```

6c · Find the upload's INTENSITY dataset

Every upload produces an INTENSITY dataset to start — input for downstream pipelines.

In [10]:
```
_ds = call(
    f"client.datasets.find_initial_dataset({upload_id!r})",
    client.datasets.find_initial_dataset, upload_id,
)
intensity_dataset_id = str(_ds.id) if _ds is not None else DRY["intensity_da
print(f"  intensity_dataset_id = {intensity_dataset_id}")
```

```
[dry-run] client.datasets.find_initial_dataset('<upload-uuid>')
  intensity_dataset_id = <intensity-uuid>
```

6d · Normalisation & imputation

Median-centred normalisation and missing-not-at-random imputation, run as a managed pipeline. Output is a new INTENSITY dataset.

In [11]:
```
from md_python.models import NormalisationImputationDataset

ni = NormalisationImputationDataset(
    dataset_name="NI \u2014 Demo run",
    input_dataset_ids=[intensity_dataset_id],
    normalisation_method="median",
    imputation_method="mnar",
    entity_type="protein",
)
ni_dataset_id = call(
    "client.datasets.create(ni)  \u2192  ni_dataset_id",
    client.datasets.create, ni,
) or DRY["ni_dataset_id"]
print(f"  ni_dataset_id = {ni_dataset_id}")
```

```
[dry-run] client.datasets.create(ni)  →  ni_dataset_id
  ni_dataset_id = <ni-output-uuid>
```

6e · Wait for the normalised dataset

In [12]:
```
call(
    f"client.datasets.wait_until_complete({upload_id!r}, {ni_dataset_id!r})"
    client.datasets.wait_until_complete, upload_id, ni_dataset_id,
)
```

```
[dry-run] client.datasets.wait_until_complete('<upload-uuid>', '<ni-output-u
uid>')
```

6f · Pairwise comparison (limma)

Differential expression between conditions, all contrasts in one run so FDR correction sees the full picture.

In [13]:
```
from md_python.models import PairwiseComparisonDataset
```

```
pc = PairwiseComparisonDataset(
    dataset_name="Pairwise \u2014 Demo run",
    input_dataset_ids=[ni_dataset_id],
    sample_metadata=upload.sample_metadata,
    condition_column="condition",
    condition_comparisons=[["treatment", "control"]],
    entity_type="protein",
)
pairwise_dataset_id = call(
    "client.datasets.create(pc)  \u2192  pairwise_dataset_id",
    client.datasets.create, pc,
) or DRY["pairwise_dataset_id"]
print(f"  pairwise_dataset_id = {pairwise_dataset_id}")

call(
    f"client.datasets.wait_until_complete({upload_id!r}, {pairwise_dataset_i
    client.datasets.wait_until_complete, upload_id, pairwise_dataset_id,
)
```

```
[dry-run] client.datasets.create(pc)  →  pairwise_dataset_id
  pairwise_dataset_id = <pairwise-uuid>
[dry-run] client.datasets.wait_until_complete('<upload-uuid>', '<pairwise-uu
id>')
```

6g · ANOVA

Omnibus F-test across three or more conditions. Same input as pairwise; same waiting pattern. We use `MinimalDataset` since ANOVA reuses the generic dataset surface with `job_slug='anova'`.

In [14]:

```
from md_python.models import MinimalDataset

anova = MinimalDataset(
    dataset_name="ANOVA \u2014 Demo run",
    input_dataset_ids=[ni_dataset_id],
    job_slug="anova",
    job_run_params={
        "sample_metadata": upload.sample_metadata,
        "condition_column": "condition",
        "comparisons_type": "all",
        "entity_type": "protein",
    },
)
anova_dataset_id = call(
    "client.datasets.create(anova)  \u2192  anova_dataset_id",
    client.datasets.create, anova,
) or DRY["anova_dataset_id"]
print(f"  anova_dataset_id = {anova_dataset_id}")

call(
    f"client.datasets.wait_until_complete({upload_id!r}, {anova_dataset_id!r
    client.datasets.wait_until_complete, upload_id, anova_dataset_id,
)

print("\nfull pipeline complete:")
```

```
print(f"  upload          {upload_id}")
print(f"  intensity       {intensity_dataset_id}")
print(f"  normalised      {ni_dataset_id}")
print(f"  pairwise (DEA)  {pairwise_dataset_id}")
print(f"  anova           {anova_dataset_id}")
```

```
[dry-run] client.datasets.create(anova)  →  anova_dataset_id
  anova_dataset_id = <anova-uuid>
[dry-run] client.datasets.wait_until_complete('<upload-uuid>', '<anova-uuid
>')

full pipeline complete:
  upload          <upload-uuid>
  intensity       <intensity-uuid>
  normalised      <ni-output-uuid>
  pairwise (DEA)  <pairwise-uuid>
  anova           <anova-uuid>
```

---

7 · BETA — Programmatic dashboards

We're experimenting with a new API surface that lets you **build dashboards from code** — workspaces, tabs, and visualisation modules — the same dashboards you can build in the UI, but reproducible, scriptable, and reviewable.

Still beta — the wire format is being polished and a couple of fields still need hand-crafting — but the bones are here. Below we'll create a workspace, add a tab, and place three real visualisations on the CKD demo dataset.

7a · Browse the module catalogue

Every visualisation in the app is in the registry — heatmaps, volcano plots, dose-response curves, PCA, dataset tables. Each declares its full parameter schema.

In [15]:
```
modules = client.module_registry.list()
print(f"{len(modules)} module types available\n")

from collections import defaultdict
by_group = defaultdict(list)
for m in modules:
    by_group[m.group].append(m.id)
for group, ids in sorted(by_group.items()):
    print(f"  {group}")
    for mid in ids:
        print(f"      {mid}")
```

```
42 module types available

  ANOVA
      anova_volcano_plot
  Dose–response analysis
      dose_response_curve_plot
      dose_response_volcano_plot
      summarised_dataset_table
  Experiment
      dimensionality_reduction_plot
      dot
      entity_abundance_plot
      line_plot
      roc_curve
      violin
  Gene set enrichment analysis
      gsea_dot_plot
  General
      dataset_table
      entity_mapping_graph
      heading
      page_break
      qc_summary_table
      text
  Heatmap
      dendrogram_plot
      heatmap
      pairwise_heatmap
  Over representation analysis
      ppi_network_graph
      reactome_ora_bar
      reactome_ora_strip
      reactome_ora_table
  Pairwise analysis
      list_table
      log_log_plot
      pairwise_volcano_plot
      peptide_fold_change_plot
      pvalues_histogram
  Protein list
      upset_plot
  Quality control
      box_plot
      cv_distribution_plot
      cv_distribution_violin_plot
      entity_detection_coverage_plot
      entity_rank_plot
      instrument_qc_bar_chart
      intensity_distribution_plot
      missing_values_by_feature_plot
      missing_values_by_sample_plot
      missing_values_heatmap
      ptm_intensity_scatter
      quality_control_report_classic
```

7b · Inspect one module's parameters

`heatmap` is a good example — a dozen tunable parameters, each with a registry default.

In [16]:
```
heatmap = client.module_registry.get("heatmap")
print(f"name        : {heatmap.name}")
print(f"setting keys: {heatmap.setting_keys()[:8]}...")
print(f"defaults    : {heatmap.defaults()}")
```

```
name        : Heatmap
setting keys: ['datasetsSearch', 'entityType', 'dataType', 'normalisationMet
hod', 'correlationMethod', 'correlationAxis', 'proteinListSearch', 'sampleNa
mes']...
defaults    : {'dataType': 'abundance', 'normalisationMethod': 'zscore', 'co
rrelationMethod': 'pearson', 'correlationAxis': 'samples', 'proteinListSearc
h': {'proteinListId': None, 'proteinListData': None, 'proteins': []}, 'sampl
eNames': {'values': []}, 'clusterByFeatures': True, 'clusterBySamples': Tru
e, 'nClustersFeatures': 2, 'nClustersSamples': 2, 'clusteringMethod': 'hiera
rchical', 'hierarchicalLinkageMethod': 'ward', 'hierarchicalDistanceMetric':
'euclidean', 'maxLeaves': 1000, 'plotFeaturesDendrograms': True, 'plotSample
sDendrograms': True, 'showAnnotations': False, 'showSampleNamesCorrelationHe
atmap': False, 'annotationField': 'gene_name', 'sampleMetadataColOrder': [],
'sampleInformation': [], 'titleDisplay': 'default', 'plotSize': {'fixed': Fa
lse}}
```

7c · Build the dataset envelope

The wire format for a dataset binding is an envelope (not a bare id) — it includes the dataset name and its upload id. The helper below builds it; this is what's being polished into the client.

In [17]:
```
def dataset_envelope(dataset_id: str, dataset_name: str, upload_id: str, dat
    """Build the persisted datasetsSearch envelope for a single-dataset modu
    return {
        "type": dataset_type,
        "individualResults": [
            {"id": dataset_id, "name": dataset_name, "experimentId": upload_
        ],
        "searchResult": None,
        "liveUpdate": False,
        "keywords": [dataset_name],
    }

envelope = dataset_envelope(
    dataset_id=DATASET_ID,
    dataset_name=ds.name,
    upload_id=UPLOAD_ID,
    dataset_type="INTENSITY",
)
print(json.dumps(envelope, indent=2))
```

```
{
  "type": "INTENSITY",
  "individualResults": [
    {
      "id": "1e97e2ae-1ad6-4d3b-a772-f2d496d152fa",
      "name": "CKD Demo 2025",
      "experimentId": "6db6e841-a1ef-4d74-b920-c6be7b581fb1"
    }
  ],
  "searchResult": null,
  "liveUpdate": false,
  "keywords": [
    "CKD Demo 2025"
  ]
}
```


7d · Create a workspace and a tab

In [18]:
```
import time

workspace = client.workspaces.create(
    name=f"API Tour \u2014 CKD Demo ({int(time.time())})",
    description="Built from the API tour notebook.",
)
tab = client.workspaces.tabs.create(str(workspace.id), name="Quality control
print(f"workspace : {workspace.id}")
print(f"tab       : {tab.id}")
```

```
workspace : c0165747-5e5e-4039-8484-dbcafc71bcfe
tab       : 08ed47ce-bed8-425d-a251-201227336cd1
```

7e · Place three visualisation modules

Each `create_with_defaults` call fills in every registry-declared default automatically — you only specify what you want to override. The grid is 12 columns wide.

In [19]:
```
common = {"datasetsSearch": envelope, "entityType": "protein"}

# Module 1 — Missing values heatmap (top, full width)
mvh = client.workspaces.modules.create_with_defaults(
    workspace_id=str(workspace.id), tab_id=str(tab.id),
    item_id="missing_values_heatmap",
    x=0, y=0, width=12, height=12, settings=common,
)
print(f"missing_values_heatmap   placed \u2192 {mvh.id}")

# Module 2 — Dimensionality reduction (left, second row)
pca = client.workspaces.modules.create_with_defaults(
    workspace_id=str(workspace.id), tab_id=str(tab.id),
    item_id="dimensionality_reduction_plot",
    x=0, y=12, width=6, height=12, settings=common,
)
print(f"dimensionality_reduction \u2192 {pca.id}")
```

```
# Module 3 — CV distribution (right, second row)
cv = client.workspaces.modules.create_with_defaults(
    workspace_id=str(workspace.id), tab_id=str(tab.id),
    item_id="cv_distribution_plot",
    x=6, y=12, width=6, height=12,
    settings={**common, "xAxis": [{"field": "sample_name", "order": "none"}]
)
print(f"cv_distribution_plot     \u2192 {cv.id}")
```

```
missing_values_heatmap   placed → 77c6c6d2-82cf-423e-b674-5f05a8bb8168
dimensionality_reduction → 6fbdda10-383b-4c29-8fd5-fbef375f0d03
cv_distribution_plot     → c14cb996-5c37-4103-9014-816d85f2257b
```

7f · Open the workspace in the app

Click through. The dashboard is live, the plots are rendering against the same dataset we explored in section 4.

In [20]:
```
app_base = client.base_url.replace("/api", "")
url = f"{app_base}/workspaces/{workspace.id}"
print(f"\n  open in browser: {url}\n")
```

```
  open in browser: https://dev.massdynamics.com/workspaces/c0165747-5e5e-4039-8484-dbcafc71bcfe
```

8 · Cleanup

Delete the demo workspace when you're done — cascades through tabs and modules. Comment out to keep the workspace between recordings.

In [21]:
```
client.workspaces.delete(str(workspace.id))
print(f"deleted workspace {workspace.id}")
```

```
deleted workspace c0165747-5e5e-4039-8484-dbcafc71bcfe
```